Course Name:	**ID**
Programming In Java	*JAVA-IV-T41*

PROJECT AND TEAM INFORMATION

Project Title

Multiplayer Game Server Monitoring and Anti-Cheat System

Student Information

Name	
Name: Rana, Rupali **ID: 240221023** **Email: 240221023@geu.ac.in**	

Course Name: *Programming In Java*

ID *JAVA-IV-T41*

PROJECT ABSTRACT

MOTIVATION

This project focuses on developing a simplified server-side anti-cheat monitoring system for multiplayer games using Java. The system simulates multiple players performing in-game actions such as movement and interactions, which are processed by a centralized server. The server analyzes these actions using predefined rule-based logic to detect abnormal behavior such as unrealistic speed, excessive actions, or invalid operations.

Each player is assigned a dynamic risk score based on their activity. Based on this score, players are classified into categories such as Safe, Suspicious, or Blocked. The system does not directly penalize players but assists moderators by identifying potentially unfair behavior.

The project demonstrates key Java concepts including object-oriented programming, multithreading, synchronization, collections, exception handling, and GUI development . It aims to provide a simple, privacy-friendly, and effective alternative to complex anti-cheat systems used in large-scale gaming platforms.

UPDATED PROJECT APPROACH AND ARCHITECTURE

The system follows a modular architecture consisting of action submission, server processing, rule evaluation, and a monitoring interface. Actions are submitted manually through the Swing GUI or queued for background processing by the GameServer thread.

These actions are sent to a centralized server module and stored in a thread-safe queue to ensure synchronization. The server processes each action sequentially and applies predefined cheat detection rules sucha as speed limits, action frequency checks, and invalid behavior detection.

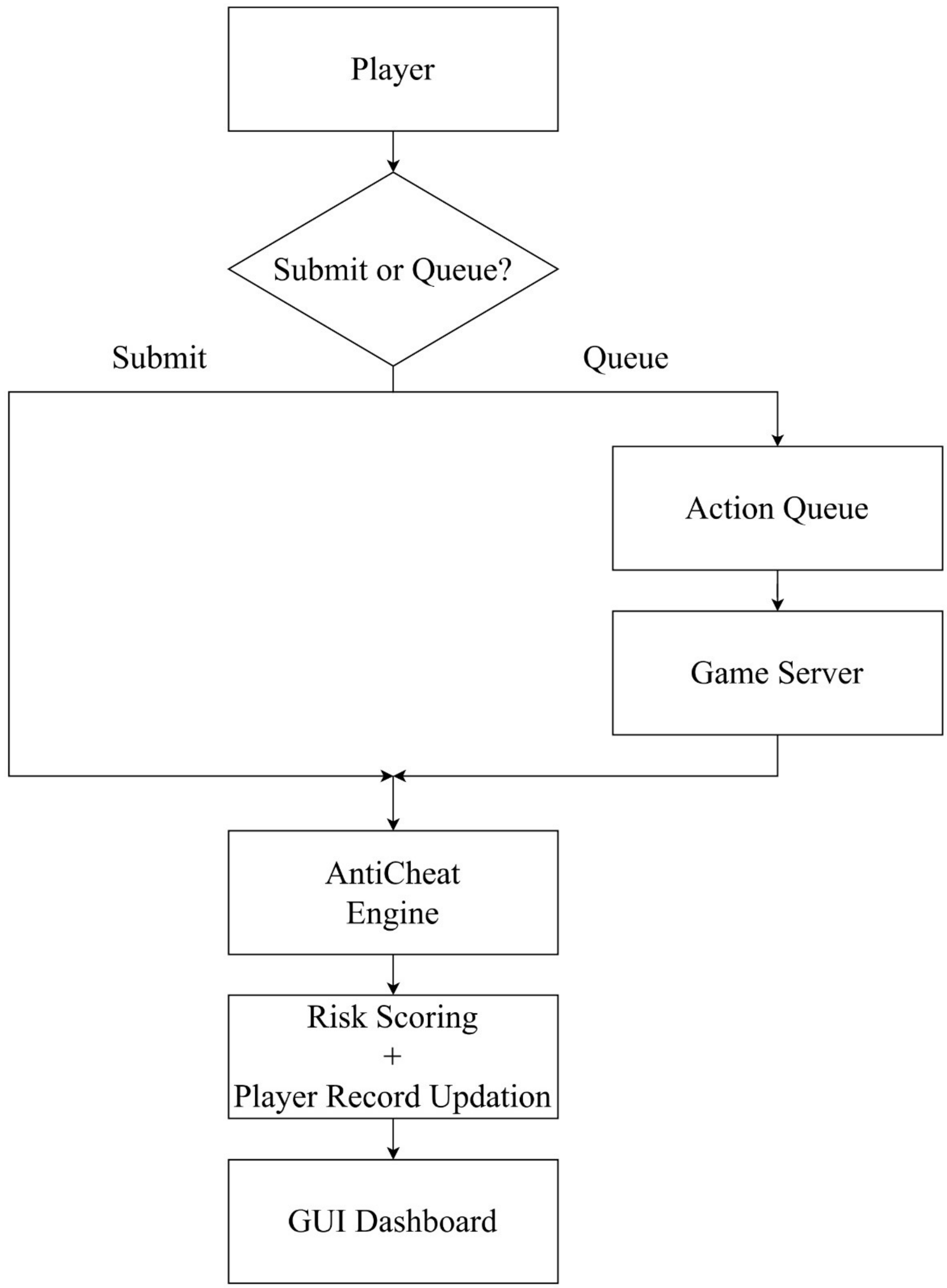
A risk scoring mechanism updates player status dynamically. The system also includes logging functionality to record suspicious activities. A Java Swing GUI displays real-time player status, risk scores, violation details, and live-editable detection thresholds.

The implementation uses Java Collections (lists, maps) for data handling, a BlockingQueue for thread-safe action processing, and file-based storage for violation logs. The codebase is maintained on GitHub for version control.

Challenges/Roadblocks

- One major challenge faced was managing multithreading and ensuring thread safety while multiple players were generating actions simultaneously. This was addressed by implementing synchronized data structures and using thread-safe queues.
- Another challenge was designing effective rule-based detection without making the system overly complex. Balancing accuracy and simplicity required careful selection of rules such as speed thresholds and action frequency limits.
- Debugging concurrent execution issues and ensuring consistent server processing was also difficult. This was handled by testing individual

SYSTEM ARCHITECTURE



ASSUMPTIONS

- All players generate actions within a predefined simulation environment.
- Rule thresholds (such as speed limits or action frequency) are predefined
- The system runs in a local environment without real network communication.
- Players do not directly modify server-side logic.
- Risk classification is based only on defined rules.

Progress Overview

The project is complete. Core modules including action handling, rule-based cheat detection, risk scoring, violation logging, and the Java Swing GUI have all been successfully implemented and integrated.

Project Outcomes/Deliverables

The project delivers a working multiplayer game anti-cheat engine capable of monitoring player actions in real time. It detects suspicious behavior across six rule categories and assigns dynamic risk scores to players.

The Java Swing GUI displays live player status, colour-coded risk levels, per-player violation details, and a timestamped event log. All detection thresholds are editable at runtime. The project demonstrates multithreading, synchronization, collections, exception handling, file I/O, and GUI development in Java.

Codebase Information

The project is maintained using GitHub for version control and collaboration.

- Repository:
- Branch: main
- Important commits include implementation of cheat detection rules, risk scoring system, violation logger, and Java Swing GUI.

Testing and Validation Status

Test Type	Status	Notes
Action Queue Test	Pass	BlockingQueue enqueue and dequeue working correctly across threads
Rule Detection Test	Pass	Rules correctly detect abnormal behavior
Risk Scoring Test	Pass	Scores updated dynamically
Integration Test	Pass	All the modules are successfully integrated
GUI Testing	Pass	Swing GUI built with dark theme; displays live player table, violation detail, event log, and live-editable thresholds